Week 9 Meeting 2: Exploring VQE and Molecular Physics

Student Lab Handout

Session Overview

Topic: Understanding VQE results through physical and algorithmic exploration

Duration: 1h15

Session Objectives

- Construct a potential energy curve for the H_2 molecule
- Identify the physical origin of molecular bonding
- Evaluate how algorithmic choices affect VQE performance
- Compare variational results with exact solutions
- Distinguish physical effects from numerical artifacts

Timed Agenda (75 minutes)

1. **0:00-0:10**: Warm-up and instructions
2. **0:10-0:35**: Task A: Potential energy curve
3. **0:35-0:55**: Task B: Ansatz and optimizer exploration
4. **0:55-1:10**: Task C: Benchmarking and interpretation
5. **1:10-1:15**: Wrap-up questions

1 Introduction

In the previous lab, we used the Variational Quantum Eigensolver (VQE) to compute the ground-state energy of the hydrogen molecule at a fixed bond length. In this lab, we go one step further by exploring *how the energy depends on molecular geometry* and *how algorithmic choices affect the result*.

2 Task A: Potential Energy Curve of H_2

2.1 Physical Motivation

Chemical bonding arises from a competition between:

- attraction between electrons and nuclei,
- repulsion between nuclei,
- quantum-mechanical delocalization of electrons.

These effects are summarized in the *potential energy curve*, which shows the molecular energy as a function of the bond length. The equilibrium bond length corresponds to the minimum of this curve.

2.2 Instructions

```
import matplotlib.pyplot as plt
import numpy as np
```

```
from qiskit_nature.second_q.drivers import PySCFDriver
from qiskit_nature.units import DistanceUnit
from qiskit_nature.second_q.mappers import JordanWignerMapper
from qiskit_algorithms import VQE
from qiskit_algorithms.optimizers import SLSQP
from qiskit.circuit.library import n_local
from qiskit.primitives import StatevectorEstimator
```

You will compute the ground-state energy of H_2 for several different bond lengths using VQE.

1. Choose 6-8 bond lengths between 0.4 Å and 2.2 Å.
2. For each bond length:
   - rebuild the molecular Hamiltonian,
   - map it to qubits,
   - compute the ground-state energy using VQE.
3. Store and plot the resulting energies.

2.3 Skeleton Code

```
bond_lengths = np.linspace(0.4, 2.2, 8)
energies = []

for R in bond_lengths:
    driver = PySCFDriver(
        atom=f"H␣0␣0␣0;␣H␣0␣0␣{R}",
```

```
        basis="sto3g",
        unit=DistanceUnit.ANGSTROM
    )
    problem = driver.run()
    qubit_op = mapper.map(problem.hamiltonian.second_q_op())

    result = vqe.compute_minimum_eigenvalue(qubit_op)
    energy = result.eigenvalue.real + problem.nuclear_repulsion_energy
    energies.append(energy)
```

2.4 Plotting the Result

```
plt.plot(bond_lengths, energies, 'o-')
plt.xlabel("Bond length (\u212B)") # unicode
plt.ylabel("Energy (Hartree)")
plt.title("H$_2$ Potential Energy Curve (VQE)")
plt.grid()
plt.show()
```

Checkpoint You should observe:

- a clear minimum in the energy,
- an equilibrium distance near 0.7-0.8 .

3 Task B: Exploring Algorithmic Choices

VQE results depend not only on physics, but also on algorithmic design.

3.1 Changing the Ansatz

Replace the ansatz used in Lab 1 by an alternative circuit.

```
from qiskit.circuit.library import EfficientSU2
ansatz = EfficientSU2(num_qubits=4, reps=2)
```

```
print("=== New Ansatz (EfficientSU2) ===")
display(ansatz.draw('mpl'))

print("Depth:", ansatz.depth())
print("Parameters:", ansatz.num_parameters)
```

Repeat one or two energy calculations and compare with your previous results.

Questions

- Does the energy decrease?
- Is convergence faster or slower?

3.2 Changing the Optimizer

Replace the optimizer by an alternative classical method.

```
from qiskit_algorithms.optimizers import COBYLA
optimizer = COBYLA(maxiter=200)
```

Questions

- Is the optimization more stable?
- Does it reach the same minimum energy?

4 Task C: Benchmarking and Interpretation

4.1 Exact Diagonalization

Compare your VQE result with an exact classical solution.

```
from qiskit_algorithms import NumPyMinimumEigensolver
exact = NumPyMinimumEigensolver()
exact_result = exact.compute_minimum_eigenvalue(qubit_op)
```

Questions

- How close is VQE to the exact energy?
- Where do discrepancies come from?

5 Discussion and Wrap-Up

Answer briefly:

- Which part of this lab was quantum?
- Which part was classical?
- What currently limits the accuracy of VQE the most?